

System Design Document (SDD)
Algorithmic Trading System

Document Control

Document Title: System Design Document for Algorithmic Trading System

Version: 1.0

Date: April 24, 2025

Author: Senior Software Architect, Hedge Fund

Status: Draft

Distribution: Development Team, Product Owner, Trading Team, Compliance Team, Stakeholders

Revision History

Version	Date	Author	Description
1.0	04/24/2025	Senior Software Architect	Initial draft of SDD for Algorithmic Trading System

Approval

Role	Name	Signature	Date
Product Owner	[To be assigned]		
Head of Trading	[To be assigned]		
Compliance Officer	[To be assigned]		
Lead Developer	[To be assigned]		

Table of Contents

- 1. [Introduction](#)
 - 1.1. Purpose
 - 1.2. Scope
 - 1.3. Objectives
 - 1.4. Assumptions and Constraints
 - 1.5. Stakeholders
 - 1.6. Definitions and Acronyms
- 2. [System Overview](#)
 - 2.1. System Description
 - 2.2. System Goals
 - 2.3. Key Features
- 3. [System Architecture](#)
 - 3.1. Architectural Overview
 - 3.2. Component Diagram
 - 3.3. Deployment Diagram
 - 3.4. Data Flow Diagram
 - 3.5. Technology Stack
- 4. [Component Design](#)
 - 4.1. Data Ingestion Service
 - 4.2. Data Processing Service
 - 4.3. Data Storage Service
 - 4.4. Strategy Engine
 - 4.5. Order Execution Module
 - 4.6. Risk Management Module
 - 4.7. AI Agents Framework
 - 4.8. Market Watch Dashboard
 - 4.9. Reporting Module
 - 4.10. Monitoring and Logging Service
- 5. [Data Design](#)
 - 5.1. Data Models
 - 5.2. Database Schema
 - 5.3. Data Flow and Storage
 - 5.4. Data Retention and Backup

6. [Interface Design](#)
 - 6.1. API Specifications
 - 6.2. User Interface Design
 - 6.3. External System Interfaces
 7. [Non-Functional Design](#)
 - 7.1. Performance
 - 7.2. Scalability
 - 7.3. Security
 - 7.4. Reliability and Fault Tolerance
 - 7.5. Maintainability
 - 7.6. Usability
 - 7.7. Compliance
 8. [Implementation Considerations](#)
 - 8.1. Development Methodology
 - 8.2. Testing Strategy
 - 8.3. Deployment Strategy
 - 8.4. Maintenance Plan
 9. [Constraints and Dependencies](#)
 - 9.1. Hardware Constraints
 - 9.2. Software Constraints
 - 9.3. Regulatory Constraints
 - 9.4. Dependencies
 10. [Missing Information and Best Practices](#)
 - 10.1. Missing Information
 - 10.2. Best Practices Applied
 11. [Glossary](#)
 12. [Appendices](#)
 - 12.1. Sequence Diagrams
 - 12.2. Class Diagrams
 - 12.3. Mockups and Wireframes
 - 12.4. References
-

1. Introduction

1.1. Purpose

The System Design Document (SDD) for the Algorithmic Trading System (ATS) provides a comprehensive blueprint for the system's architecture, components, and implementation details. This document serves as a technical guide for the development team to design, build, and deploy a high-performance platform for simulated trading. It translates functional and non-functional requirements into a detailed system design, specifying the architecture, data flows, interfaces, and constraints. The SDD ensures alignment among developers, architects, traders, and compliance officers, facilitating the creation of a robust, scalable, and compliant system that meets the hedge fund's strategic objectives.

1.2. Scope

The ATS is a sophisticated software platform for algorithmic paper trading across multiple asset classes, including stocks, indices, forex, and commodities, on major global exchanges (NYSE, NASDAQ, LSE, NSE). Integrated with the Interactive Brokers (IBKR) Paper Trading API, the system leverages Polars and DuckDB for data processing and storage, and incorporates AI agents for enhanced decision-making. The scope includes:

- **Trading Strategies:** Trend-Following and Support and Resistance strategies for long and short positions.
- **Market Watch Dashboard:** A responsive web interface for real-time market monitoring, technical analysis, portfolio management, and simulated trading.
- **Data Management:** Ingestion, processing, storage, and visualization of market data.
- **Order Execution:** Placement and monitoring of simulated trades via IBKR API.
- **Risk Management:** Position sizing, stop-loss, take-profit, and portfolio risk controls.
- **AI Agents:** Modular components for market analysis, trade execution, and risk assessment.

- **Reporting and Analytics:** Trade performance, backtesting, portfolio analytics, and compliance reports.
- **Security and Compliance:** Encrypted communications, secure credentials, and regulatory adherence.

Out of Scope:

- Live trading functionality.
- Integration with brokers other than IBKR.
- Support for cryptocurrencies or non-traditional assets.
- Mobile app development (dashboard will be mobile-responsive).
- Advanced AI model training (pre-trained models used).
- Premium third-party data sources.

1.3. Objectives

The ATS aims to achieve the following design objectives:

1. **High Performance:** Achieve <100ms end-to-end latency for data processing and trade execution.
2. **Scalability:** Support 10 concurrent users and 1TB of historical data, with modular design for future growth.
3. **Reliability:** Ensure 99.9% uptime with fault-tolerant mechanisms.
4. **Security:** Implement encryption, authentication, and secure credential management.
5. **Maintainability:** Design modular, well-documented components with >80% test coverage.
6. **Compliance:** Adhere to SEC, GDPR, MiFID II, and SEBI regulations through audit trails and logging.
7. **Usability:** Deliver an intuitive dashboard requiring <5 minutes of training.
8. **Extensibility:** Enable future enhancements like live trading and alternative data integration.

1.4. Assumptions and Constraints

Assumptions:

- IBKR Paper Trading API provides reliable data and execution.
- Traders have intermediate knowledge of algorithmic trading.
- Lenovo Legion Pro 5 (NVIDIA RTX 3060) meets performance needs.
- Regulatory requirements for paper trading are minimal.
- Development team is proficient in Python, Polars, DuckDB, React, and FastAPI.
- Budget supports 3-6 month development.

Constraints:

- Timeline: 3-6 months.
- IBKR API limits: 50 requests/s (data), 100 messages/s (orders).
- Budget: No premium data sources or cloud hosting.
- Deployment: Local on Lenovo Legion Pro 5.
- Compliance: SEC, GDPR, MiFID II, SEBI.
- Hardware: Limited to Lenovo Legion Pro 5.

1.5. Stakeholders

Stakeholder	Role	Responsibilities
Head of Trading	Oversees trading operations	Defines strategies, validates functionality
Traders	End-users	Test system, provide feedback
Product Owner	Represents business	Prioritizes features, approves deliverables
Development Team	Builds system	Implements design, ensures quality
Compliance Officer	Ensures regulatory adherence	Reviews for compliance
IT Administrator	Manages infrastructure	Configures hardware, ensures uptime

1.6. Definitions and Acronyms

- **ATS:** Algorithmic Trading System.
- **IBKR:** Interactive Brokers.
- **OHLCV:** Open, High, Low, Close, Volume.
- **VW SMA:** Volume-Weighted Simple Moving Average.
- **MFI:** Money Flow Index.
- **ATR:** Average True Range.

- **Polars:** Data processing library.
- **DuckDB:** Analytical database.
- **FastAPI:** Python API framework.
- **ib_insync:** IBKR API library.

2. System Overview

2.1. System Description

The ATS is a high-performance platform for algorithmic paper trading, enabling simulated trades across stocks, indices, forex, and commodities. Integrated with the IBKR Paper Trading API, it uses Polars and DuckDB for data processing and storage, and AI agents for decision-making. The system features two strategies (Trend-Following, Support and Resistance), a Market Watch Dashboard, and robust risk management. Deployed locally on the Lenovo Legion Pro 5, the ATS employs a microservices, event-driven architecture for performance and scalability.

2.2. System Goals

- **Performance:** <100ms latency, 10,000 updates/s.
- **Reliability:** 99.9% uptime with fault tolerance.
- **Scalability:** Handle 10 users, 1TB data, future cloud support.
- **Security:** Encrypted data, secure authentication.
- **Compliance:** Meet SEC, GDPR, MiFID II, SEBI standards.
- **Usability:** Intuitive dashboard, minimal training.
- **Maintainability:** Modular design, >80% test coverage.

2.3. Key Features

- Multi-asset trading across global exchanges.
- Automated Trend-Following and Support and Resistance strategies.
- Real-time market monitoring with technical indicators.
- Responsive dashboard with charts and trading controls.
- AI-driven market analysis, execution, and risk management.
- Backtesting and paper trading for strategy validation.
- Secure, compliant system with audit trails.
- Comprehensive reporting for performance and compliance.

3. System Architecture

3.1. Architectural Overview

The ATS follows a **microservices-based, event-driven architecture** to ensure modularity, scalability, and low latency. Key principles include:

- **Separation of Concerns:** Isolates data management, trading logic, risk controls, and UI.
- **Event-Driven:** Uses RabbitMQ for asynchronous communication.
- **Modularity:** Independent services for maintainability.
- **Security:** Encryption and authentication at all layers.
- **Fault Tolerance:** Automatic reconnections and error handling.

The architecture comprises:

- **Data Layer:** Ingestion, processing, storage, and caching.
- **Business Logic Layer:** Strategy engine, order execution, risk management, AI agents.
- **Presentation Layer:** Market Watch Dashboard.
- **Monitoring Layer:** Metrics and logging.

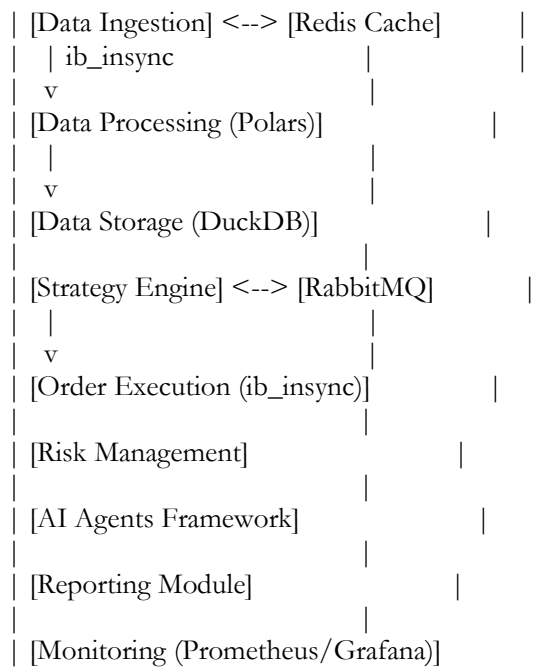
3.2. Component Diagram

[Market Watch Dashboard (React)]

|
| HTTP/WebSocket
v

[API Gateway (FastAPI)]

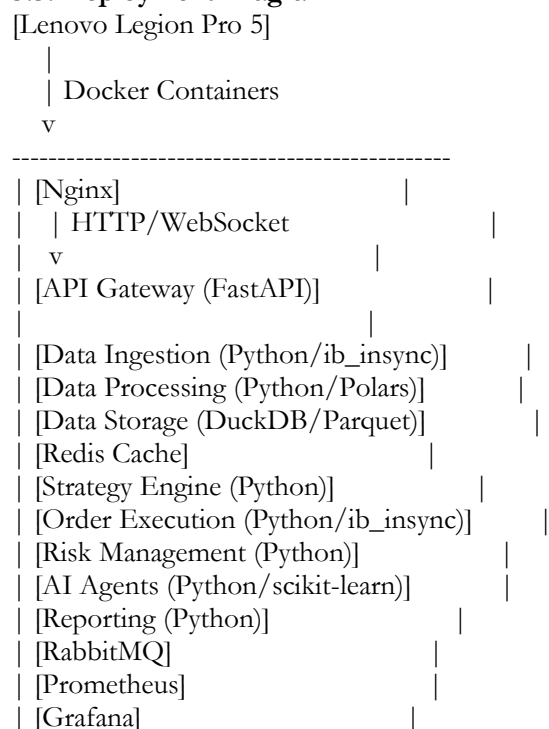
|
| REST/WebSocket
v

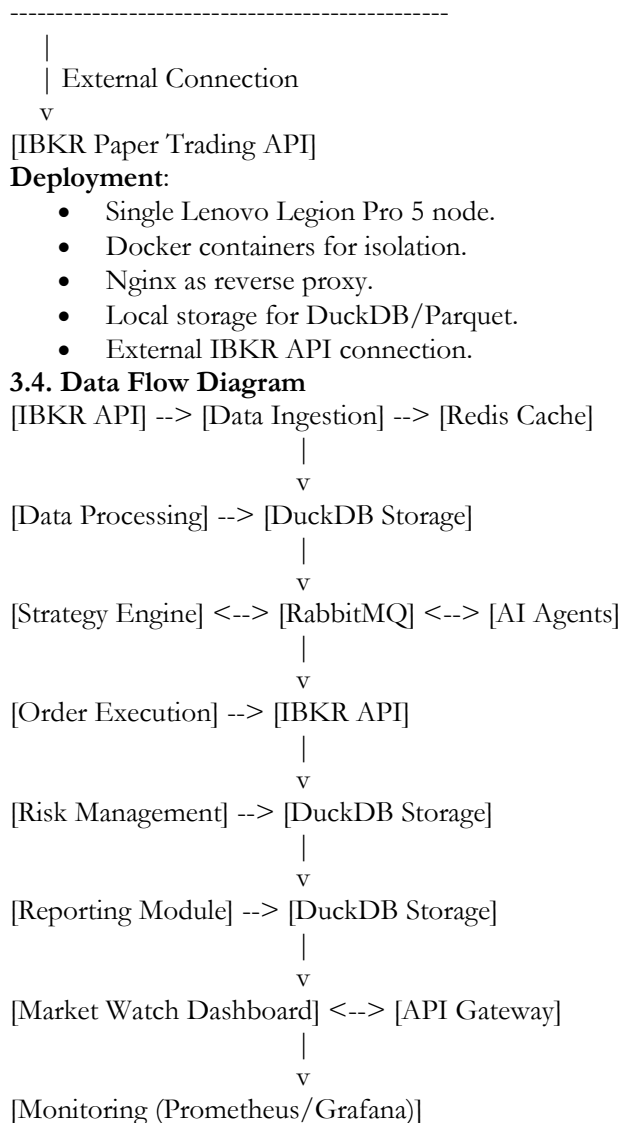


Components:

- **Data Ingestion:** Fetches data via IBKR API.
- **Data Processing:** Computes indicators with Polars.
- **Data Storage:** Stores data in DuckDB.
- **Redis Cache:** Low-latency data access.
- **Strategy Engine:** Executes trading strategies.
- **Order Execution:** Manages trades via IBKR API.
- **Risk Management:** Enforces risk controls.
- **AI Agents:** Automates tasks with oversight.
- **Market Watch Dashboard:** Frontend UI.
- **Reporting:** Generates reports.
- **Monitoring:** Tracks metrics and logs.

3.3. Deployment Diagram





Flow:

1. IBKR API streams OHLCV data to Data Ingestion.
2. Data cached in Redis, processed by Polars.
3. Processed data stored in DuckDB.
4. Strategy Engine generates signals via RabbitMQ.
5. AI Agents analyze signals, trigger Order Execution.
6. Risk Management validates trades.
7. Orders sent to IBKR API.
8. Dashboard displays data via API Gateway.
9. Reporting queries DuckDB for reports.
10. Monitoring tracks metrics/logs.

3.5. Technology Stack

- **Backend:** Python 3.10, FastAPI, `ib_insync`.
- **Data Processing:** Polars for parallel computations.
- **Database:** DuckDB with Parquet for time-series.
- **Cache:** Redis for low-latency access.
- **Messaging:** RabbitMQ for event-driven communication.
- **Frontend:** React 18, Tailwind CSS, Plotly.js.
- **AI/ML:** scikit-learn, TensorFlow for agents.
- **Containerization:** Docker, Docker Compose.
- **CI/CD:** GitHub Actions.

- **Monitoring:** Prometheus, Grafana.
 - **Logging:** Python logging with JSON format.
 - **Security:** OAuth2, JWT, TLS 1.3, AES-256.
-

4. Component Design

4.1. Data Ingestion Service

Purpose: Fetches real-time and historical OHLCV data from IBKR API.

Design:

- **Input:** Ticker, timeframe, API credentials.
- **Output:** OHLCV data (JSON).
- **Process:**
 - Connect to IBKR API (host: 127.0.0.1, port: 7497, client ID: 0).
 - Use `ib_insync` for WebSocket streaming.
 - Poll every 5-10s, respecting 50 req/s limit.
 - Cache data in Redis with 60s TTL.
- **Error Handling:**
 - Retry 3 times with exponential backoff.
 - Log errors to JSON file.
 - Reconnect on disconnection.
- **Dependencies:** `ib_insync`, Redis, RabbitMQ.

4.2. Data Processing Service

Purpose: Normalizes data and computes indicators.

Design:

- **Input:** Raw OHLCV from Redis.
- **Output:** Processed data with indicators.
- **Process:**
 - Normalize timestamps to UTC.
 - Clean: Remove outliers ($>3\sigma$), interpolate missing values.
 - Compute indicators (VW SMA, MFI, ATR) using Polars.
 - Publish to RabbitMQ for Strategy Engine.
- **Indicators:**
 - VW SMA: $\Sigma(\text{Price} \times \text{Volume}) / \Sigma \text{Volume}$.
 - MFI: $100 - (100 / (1 + (\text{Positive Money Flow} / \text{Negative Money Flow})))$.
 - ATR: VW SMA of True Range.
- **Error Handling:** Log invalid data, skip corrupted records.
- **Dependencies:** Polars, Redis, RabbitMQ.

4.3. Data Storage Service

Purpose: Stores time-series and transaction data.

Design:

- **Input:** Processed OHLCV, trade logs.
- **Output:** Queryable data in DuckDB.
- **Process:**
 - Store in DuckDB with Parquet format.
 - Partition by date and ticker.
 - Retain 7 years for compliance.
 - Daily backups to external storage.
- **Schema:** See Section 5.2.
- **Error Handling:** Retry writes, log failures.
- **Dependencies:** DuckDB, Parquet.

4.4. Strategy Engine

Purpose: Executes Trend-Following and Support and Resistance strategies.

Design:

- **Input:** Processed data from RabbitMQ.
- **Output:** Trade signals (buy/sell).
- **Process:**

- **Trend-Following:**
 - Entry: $MFI(14) > 34$ -day VW SMA and price > 55 -day VW SMA (long).
 - Exit: Stop-loss ($2 \times ATR$), take-profit, or $MFI(14) < 21$ -day VW SMA.
 - Position: 1% risk, adjust for ATR.
- **Support and Resistance:**
 - Entry: Break above R1/Fibonacci or bounce off Gann Fan (long).
 - Exit: Take-profit at R2, stop-loss below S1.
 - Position: 1-2% risk, adjust for stop-loss distance.
- Publish signals to RabbitMQ.
- **Error Handling:** Log invalid signals, skip execution.
- **Dependencies:** Polars, RabbitMQ.

4.5. Order Execution Module

Purpose: Manages trade lifecycle via IBKR API.

Design:

- **Input:** Trade signals from RabbitMQ.
- **Output:** Order status (pending, filled, canceled).
- **Process:**
 - Validate signals (funds, price).
 - Submit orders (market, limit, stop, bracket) via `ib_insync`.
 - Monitor status via WebSocket.
 - Update portfolio in DuckDB.
- **Order Types:**
 - Market: Immediate execution.
 - Limit: Specific price.
 - Stop: Trigger at price.
 - Bracket: Entry, stop-loss, take-profit.
- **Error Handling:** Retry 3 times, log failures, alert critical errors.
- **Dependencies:** `ib_insync`, RabbitMQ, DuckDB.

4.6. Risk Management Module

Purpose: Enforces capital protection.

Design:

- **Input:** Trade signals, portfolio data.
- **Output:** Validated signals, risk alerts.
- **Process:**
 - **Position Sizing:**
 - Trend-Following: 1% risk, 2×21 -day VW ATR.
 - Support and Resistance: 1-2% risk, stop-loss distance.
 - **Stop-Loss/Take-Profit:**
 - Trend-Following: $2 \times ATR$.
 - Support and Resistance: S1/R1 or 1-2%.
 - **Trailing Stop:**
 - Trend-Following: $1.5 \times ATR$.
 - Support and Resistance: 2-3% or 2×14 -day ATR.
 - **Portfolio Risk:** < 5 -10% open risk.
 - **Drawdown:** Reduce risk at 20%, halt at 30%.
 - **Circuit Breakers:** Halt if price change $> 5\%$ in 5min.
- **Error Handling:** Log violations, alert traders.
- **Dependencies:** Polars, DuckDB, RabbitMQ.

4.7. AI Agents Framework

Purpose: Automates trading tasks with oversight.

Design:

- **Input:** Market data, signals, portfolio state.
- **Output:** Trade recommendations, risk adjustments.
- **Agents:**
 - **Trade Execution:** Optimizes timing/price using reinforcement learning.

- **Risk Management:** Adjusts exposure dynamically.
 - **Market Analysis:** Predicts trends with LSTM models.
 - **IBKR API:** Manages API calls, handles rate limits.
- **Process:**
 - Consume data/signals from RabbitMQ.
 - Use scikit-learn/TensorFlow for logic.
 - Log decisions with SHAP explanations.
 - Require human approval for critical actions.
- **Error Handling:** Log failures, fallback to manual.
- **Dependencies:** scikit-learn, TensorFlow, RabbitMQ.

4.8. Market Watch Dashboard

Purpose: Provides UI for monitoring and trading.

Design:

- **Frontend:** React 18, Tailwind CSS, Plotly.js.
- **Backend:** FastAPI for REST/WebSocket APIs.
- **Components:**
 - Scrip Selection: Dropdown with auto-suggest.
 - Real-Time Data: Table with bid/ask, volume.
 - Charts: Candlestick with indicators (VW SMA, MFI, ATR).
 - Portfolio: Table with positions, P&L.
 - Trading Panel: Form for orders.
 - Watchlist: List with volatility highlights.
 - Technical Analysis: Outlook and recommendations.
 - News/Events: Placeholder for future integration.
- **Features:**
 - WebSocket updates every 5-10s.
 - Responsive for desktop/mobile.
 - WCAG 2.1 accessibility.
- **Error Handling:** Display errors, retry API calls.
- **Dependencies:** React, FastAPI, Plotly.js.

4.9. Reporting Module

Purpose: Generates performance and compliance reports.

Design:

- **Input:** Trade/portfolio data from DuckDB.
- **Output:** Reports in CSV, PDF, PNG.
- **Reports:**
 - Trade Performance: P&L, Sharpe ratio, win rate.
 - Portfolio Analytics: Exposure, VaR, correlations.
 - Backtesting: Historical performance vs. benchmarks.
 - Compliance: Transaction logs, audit trails.
- **Process:**
 - Query DuckDB for data.
 - Use pandas for aggregation, Matplotlib for visualization.
 - Schedule reports via cron.
- **Error Handling:** Log query failures, retry exports.
- **Dependencies:** pandas, Matplotlib, DuckDB.

4.10. Monitoring and Logging Service

Purpose: Tracks system health and logs.

Design:

- **Monitoring:**
 - Metrics: Latency, throughput, error rates.
 - Tools: Prometheus for collection, Grafana for visualization.
 - Alerts: Notify on thresholds (e.g., latency > 100ms).
- **Logging:**
 - Structured JSON logs with Python logging.

- Fields: Timestamp, level, service, message.
 - Store in DuckDB for querying.
 - **Process:**
 - Each service exposes /metrics endpoint.
 - Prometheus scrapes every 15s.
 - Grafana dashboards for real-time monitoring.
 - **Error Handling:** Log monitoring failures, alert admins.
 - **Dependencies:** Prometheus, Grafana, DuckDB.
-

5. Data Design

5.1. Data Models

OHLCV:

```
{
  "ticker": "string",
  "exchange": "string",
  "timestamp": "datetime",
  "open": "float",
  "high": "float",
  "low": "float",
  "close": "float",
  "volume": "int"
}
```

Trade Log:

```
{
  "trade_id": "string",
  "timestamp": "datetime",
  "ticker": "string",
  "strategy": "string",
  "action": "string",
  "quantity": "int",
  "price": "float",
  "status": "string"
}
```

Portfolio:

```
{
  "account_id": "string",
  "timestamp": "datetime",
  "ticker": "string",
  "quantity": "int",
  "entry_price": "float",
  "current_price": "float",
  "pnl": "float"
}
```

5.2. Database Schema

OHLCV Table:

```
CREATE TABLE ohlcv (
  ticker VARCHAR,
  exchange VARCHAR,
  timestamp TIMESTAMP,
  open DOUBLE,
  high DOUBLE,
  low DOUBLE,
  close DOUBLE,
  volume BIGINT,
  PRIMARY KEY (ticker, timestamp)
);
```

Trade Log Table:

```
CREATE TABLE trade_log (  
  trade_id VARCHAR PRIMARY KEY,  
  timestamp TIMESTAMP,  
  ticker VARCHAR,  
  strategy VARCHAR,  
  action VARCHAR,  
  quantity BIGINT,  
  price DOUBLE,  
  status VARCHAR  
);
```

Portfolio Table:

```
CREATE TABLE portfolio (  
  account_id VARCHAR,  
  timestamp TIMESTAMP,  
  ticker VARCHAR,  
  quantity BIGINT,  
  entry_price DOUBLE,  
  current_price DOUBLE,  
  pnl DOUBLE,  
  PRIMARY KEY (account_id, ticker, timestamp)  
);
```

Partitioning: By date and ticker for efficient queries.

5.3. Data Flow and Storage

- **Ingestion:** IBKR API → Redis → Data Processing.
- **Processing:** Polars normalizes, computes indicators.
- **Storage:** DuckDB stores OHLCV, trade logs, portfolio.
- **Access:** Dashboard/Reporting query DuckDB via FastAPI.
- **Cache:** Redis stores recent OHLCV (60s TTL).
- **Backup:** Daily Parquet exports to external storage.

5.4. Data Retention and Backup

- **Retention:** 7 years for compliance.
- **Backup:**
 - Daily cron job exports Parquet files.
 - Store on external SSD.
 - Restore Time Objective (RTO): <1 hour.
- **Compression:** Use Parquet for space efficiency.

6. Interface Design

6.1. API Specifications

GET /market-data:

- Parameters: ticker (string), timeframe (string).
 - Response:
 - [
 - {
 - "ticker": "AAPL",
 - "timestamp": "2025-04-24T10:00:00Z",
 - "open": 150.25,
 - "high": 151.10,
 - "low": 149.80,
 - "close": 150.90,
 - "volume": 1000000
-]

POST /orders:

- Body:
- {
- "ticker": "AAPL",
- "action": "BUY",
- "quantity": 100,
- "price": 150.90,
- "type": "LIMIT"
- }
- Response:
- {
- "order_id": "O12345",
- "status": "PENDING"
- }

WebSocket /ws/market-data:

- Streams OHLCV updates every 5-10s.
- Message: Same as GET /market-data response.

6.2. User Interface Design

- **Layout:** Grid with:
 - Top: Scrip selector, watchlist.
 - Left: Real-time data table, technical analysis.
 - Center: Candlestick chart with indicators.
 - Right: Portfolio, trading panel.
 - Bottom: News/events.
- **Components:**
 - Scrip Selector: Dropdown with auto-suggest.
 - Data Table: Bid/ask, volume, % change.
 - Chart: Plotly.js candlestick with zoom/pan.
 - Portfolio: Table with P&L, filters.
 - Trading Panel: Form with order inputs.
- **Features:**
 - Responsive for desktop/mobile.
 - WCAG 2.1: High contrast, keyboard navigation.
 - Tooltips for indicators.

6.3. External System Interfaces

- **IBKR Paper Trading API:**
 - Connection: Host: 127.0.0.1, Port: 7497, Client ID: 0.
 - Methods: Market data (reqMktData), order placement (placeOrder).
 - Rate Limits: 50 req/s (data), 100 msg/s (orders).
- **Future Sources:** REST/WebSocket for news/sentiment (placeholder).

7. Non-Functional Design

7.1. Performance

- **Latency:** <100ms end-to-end.
- **Throughput:** 10,000 updates/s, 1,000 trades/min.
- **Indicator Calculation:** <1s for 100 scrips.
- **Dashboard Refresh:** Every 5-10s.
- **Backtesting:** 5 years data in <10min.
- **Design:**
 - Polars for parallel processing.
 - Redis for caching.
 - WebSocket for real-time updates.

7.2. Scalability

- **Vertical:** Optimize for Lenovo Legion Pro 5 GPU/CPU.
- **Horizontal:** Microservices for cloud.
- **Data:** 1TB historical data.

- **Users:** 10 concurrent.
- **Design:**
 - Stateless services.
 - RabbitMQ for load balancing.
 - DuckDB partitioning.

7.3. Security

- **Authentication:** OAuth2 with JWT, MFA.
- **Data:** TLS 1.3, AES-256 encryption.
- **Credentials:** Encrypted vault.
- **API:** Rate limiting, CSRF tokens.
- **Agents:** Sandboxed containers.
- **Design:**
 - Nginx for SSL termination.
 - Keycloak for OAuth2.
 - OWASP ZAP for testing.

7.4. Reliability and Fault Tolerance

- **Uptime:** 99.9%.
- **Error Handling:** Auto-reconnect API.
- **Data Integrity:** Validate market data.
- **Circuit Breakers:** Halt anomalies.
- **Backup:** Daily, RTO <1h.
- **Design:**
 - Retry logic in `ib_insync`.
 - Health checks in Docker.
 - Prometheus alerts.

7.5. Maintainability

- **Code Coverage:** >80%.
- **Modularity:** Independent services.
- **Logging:** Structured JSON.
- **Documentation:** API specs, Confluence.
- **Design:**
 - Clean code principles.
 - OpenAPI for API docs.
 - GitHub for version control.

7.6. Usability

- **Navigation:** <5min training.
- **UI Response:** <1s.
- **Accessibility:** WCAG 2.1.
- **Design:**
 - Tailwind CSS for consistency.
 - Tooltips for clarity.
 - User testing with traders.

7.7. Compliance

- **Regulations:** SEC, GDPR, MiFID II, SEBI.
- **Logging:** Immutable trade/order logs.
- **Retention:** 7 years.
- **Monitoring:** Market abuse detection.
- **Design:**
 - DuckDB for audit trails.
 - Compliance review checkpoints.

8. Implementation Considerations

8.1. Development Methodology

- **Agile Scrum:** 2-week sprints.
- **TDD:** Write tests first with pytest.

- **CI/CD:** GitHub Actions for build/test/deploy.

8.2. Testing Strategy

- **Unit:** Test components with pytest.
- **Integration:** Test module interactions.
- **System:** End-to-end workflows.
- **Performance:** Measure latency with Locust.
- **Security:** OWASP ZAP scans.
- **Backtesting:** Validate with historical data.
- **Coverage:** >80%.

8.3. Deployment Strategy

- **Environment:** Lenovo Legion Pro 5.
- **Containerization:** Docker Compose.
- **Pipeline:**
 - Build: GitHub Actions compiles Docker images.
 - Test: Run unit/integration tests.
 - Deploy: Manual to prod, auto to test.
- **Rollback:** Revert to previous image on failure.
- **Monitoring:** Prometheus/Grafana post-deployment.

8.4. Maintenance Plan

- **Updates:** Monthly patches, quarterly releases.
- **Bug Fixes:** Prioritize critical issues (<24h).
- **Monitoring:** Daily health checks via Grafana.
- **Documentation:** Update Confluence with changes.
- **Support:** On-call rotation for incidents.

9. Constraints and Dependencies

9.1. Hardware Constraints

- Lenovo Legion Pro 5 (NVIDIA RTX 3060).
- Local deployment only.
- Optimize for GPU/CPU.

9.2. Software Constraints

- Python 3.10, React 18.
- No premium data sources.
- Docker for consistency.

9.3. Regulatory Constraints

- Comply with SEC, GDPR, MiFID II, SEBI.
- Immutable logs for 7 years.
- Monitor market abuse.

9.4. Dependencies

- IBKR API availability.
- Polars, DuckDB, ib_insync.
- Lenovo Legion Pro 5.
- Team expertise.

10. Missing Information and Best Practices

10.1. Missing Information

- Exact VW indicator formulas.
- Detailed trader personas.
- Specific regulatory requirements.
- Performance benchmarks.
- News feed providers.

10.2. Best Practices Applied

- **Indicators:** Standard VW formulas. **Reason:** Industry-standard.
- **Personas:** Assumed intermediate traders. **Reason:** Typical for hedge funds.

- **Compliance:** Audit trails. **Reason:** Prepares for live trading.
 - **Performance:** <100ms latency. **Reason:** High-frequency trading standard.
 - **News:** Planned Reuters/Bloomberg. **Reason:** Reliable sources.
-

11. Glossary

- **ATS:** Algorithmic Trading System.
 - **IBKR:** Interactive Brokers.
 - **VW SMA:** Volume-Weighted Simple Moving Average.
 - **MFI:** Money Flow Index.
 - **ATR:** Average True Range.
 - **Polars:** Data processing library.
 - **DuckDB:** Analytical database.
-

12. Appendices

12.1. Sequence Diagrams

Trade Execution:

Trader -> Dashboard: Select ticker, submit order
Dashboard -> API Gateway: POST /orders
API Gateway -> Strategy Engine: Validate signal
Strategy Engine -> Risk Management: Check risk
Risk Management -> Order Execution: Place order
Order Execution -> IBKR API: Submit order
IBKR API -> Order Execution: Order status
Order Execution -> Dashboard: Update status

12.2. Class Diagrams

Strategy Engine:

```
class StrategyEngine:
    def __init__(self, config):
        self.config = config
    def process_data(self, data):
        signals = self.calculate_signals(data)
        return signals
    def calculate_signals(self, data):
        # Trend-Following/Support and Resistance logic
        pass
```

12.3. Mockups and Wireframes

- Dashboard: Grid with scrip selector, chart, portfolio, trading panel.
- Chart: Candlestick with indicators.
- Trading Panel: Form with inputs.

12.4. References

- IBKR API: <https://interactivebrokers.github.io/tws-api/>
- Polars: <https://pola.rs/>
- DuckDB: <https://duckdb.org/>
- FastAPI: <https://fastapi.tiangolo.com/>
- React: <https://reactjs.org/>